ROUND 17 — CONSTITUTION V5: THE INVISIBLE LAYER

The Invisible Layer

V4 proved the cognition exists. V5 makes it disappear. The user should almost never visit Maestro.

Reviewer	Independent (Super Z, Z.ai) — acting as Principal Engineer specifying V5
Baseline	Commit d010883. V4 score: 10/10. 61 backend modules, 27 frontend files. 8 V4 cognitive organs built + wired. "Built but not applied" pattern broken.
V5 shift	V4 = cognitive organs that observe and judge. V5 = invisible intelligence that acts, allocates attention, forgets, imagines, and disappears into work. Organs become internal. UI becomes calm. Maestro visits the user.
Constitutional law	"Every release must make Maestro feel simpler, even if it becomes dramatically more intelligent internally." — the V5 litmus test.
Deliverable	9 specifications across 3 phases: (1) hide organs + add executive function, (2) add temporal/causal/narrative cognition, (3) ambient integration + institutional memory. Each has API + acceptance test + build order.

WHY V5 EXISTS

V4 proved that an organization can have cognitive organs. 8 organs were built, all wired, all returning real data, all user-visible. The "built but not applied" pattern was broken. The product reached 10/10 on the V4 axis. That is a significant engineering achievement.

V5 is different. V5 says: stop adding visible features. Double the internal intelligence. Reduce the visible UI every month. The user should almost never visit Maestro. Maestro visits the user — in Outlook, in Zoom, in GitHub, in Jira, in Slack. The cognitive organs become internal. The UI becomes calm. The product stops being a destination and becomes a companion.

The V5 litmus test for every commit: "Does this make Maestro feel simpler, even if it becomes more intelligent internally?" If yes, ship it. If it adds visible complexity without removing existing complexity, reject it. V5 is the first constitution whose primary metric is UI reduction, not capability addition.

Same discipline as V3/V4. Every specification has: the principle, the current codebase gap, exact files to create/modify, API contract, acceptance test, effort, build phase. The acceptance tests check both backend AND frontend — but in V5, the frontend check is often "the organ name does NOT appear in the UI" (invisibility is the goal).

Build order matters more than ever. Phase 1 (hide organs + add executive function) must come before Phase 2 (deeper cognition) because Phase 2 capabilities must be invisible from the start, not added visibly and then hidden. Phase 3 (ambient integration) requires Phase 1 + 2 because ambient surfaces display the cognition, not the organs.

The 9 V5 Specifications

#	Specification	Phase	Effort	What It Replaces/Hides
1	Hide Organ Names from UI	1	1 day	cognition.js exposes "Consciousness", "Skepticism", etc. → replace with calm human sentences
2	Executive Function Engine	1	3 days	Maestro advises → Maestro plans, sequences, and drafts actions
3	Attention Allocation	1	2 days	Consciousness knows where attention IS → decides where it SHOULD BE
4	Forgetting Engine	2	1.5 days	Compression compresses → Forgetting archives zero-predictive-value events
5	Imagination (Counterfactual)	2	2 days	Digital Twin runs scenarios → Causal counterfactual reasoning ("what if Legal disappeared?")
6	Causal Cognition	2	2 days	evidence_graph.py is correlational → add causal inference ("A caused B because 5 interventions
7	Temporal Trajectories (Org-wide)	2	1.5 days	time_axis.py works for 1 domain → org-wide trajectory memory ("trust has fallen for 8 weeks")
8	Institutional Memory Recall	3	2 days	Search returns documents → "When have we been here before?" returns 3 moments + lessons
9	Ambient Integration (Chrome Extension)	3	5 days	Maestro is a destination → Maestro appears in GitHub/Jira/Slack/Zoom/Outlook
TOTAL	**9 specs**	**3 phases**	**~20.5 days**	**V5: from cognitive architecture to invisible operating system**

Build order: Phase 1 (#1-#3, ~6 days) → Phase 2 (#4-#7, ~7 days) → Phase 3 (#8-#9, ~7 days). Total ~20.5 days. Phase 1 is the V5 foundation: hide what exists, add the ability to act. Phase 2 deepens cognition (causal, temporal, narrative, forgetting, imagination). Phase 3 makes it ambient (Maestro visits the user).

Phase 1 — Hide Organs, Add Executive Function

Phase 1 is the V5 foundation. The 8 V4 organs are genuine engineering, but their names are visible in the UI. V5 says: the organs become internal. The UI shows calm human sentences. Simultaneously, Maestro gains the ability to ACT — not just advise. Executive function is the organ that separates an advisor from an operating system.

SPEC #1 — Hide Organ Names from UI — replace "Consciousness" with "Your organization feels overloaded today"

V5 Principle

V5 Constitutional Law: "Every release must make Maestro feel simpler." The organ names (Consciousness, Skepticism, Wisdom, Metacognition, Principles, Compression, Identity, Curiosity) are internal architecture. Customers should never see them. The UI shows calm human sentences that embody the organ's output without naming the organ.

Current codebase gap

cognition.js lines 77, 103, 121, 137, 155, 181 expose organ names directly: "Consciousness — state vector", "Skepticism — challenged beliefs", "Wisdom — competing values synthesized", "Metacognition — thinking about thinking", "Principles — graduated wisdom", "Memory Compression". These are internal names displayed to users. V5 says: replace with human sentences. "Consciousness — state vector" → "Your organization feels overloaded today. Engineering attention is fragmented." "Skepticism — challenged beliefs" → "One assumption worth revisiting this week." "Wisdom — competing values synthesized" → "Here is what your organization usually does in this situation."

Files to create/modify

MODIFY static/js/cognition.js — replace all 6 organ-name labels with calm human sentences derived from the organ's output. MODIFY static/js/learn.js — replace "Identity" label with "Who your organization is" (already done partially). MODIFY static/js/today.js — replace "Curiosity" label with "One thing nobody has investigated yet" (already done partially). MODIFY static/css/invisible-maestro.css — remove any styling that makes organ sections look like distinct panels (they should feel like a continuous narrative). DO NOT change the backend APIs — the organs still return the same data. Only the display layer changes.

API contract

No new API. This is a frontend-only change. The acceptance test is: grep all JS files for organ names in user-facing strings (innerHTML, textContent, template literals). The count must be 0 for: "Consciousness", "Skepticism", "Wisdom", "Metacognition", "Principles", "Compression", "Identity", "Curiosity" — when used as user-facing labels (not in comments, variable names, or API paths).

Acceptance test (auditor will run)

1) Auditor greps all JS files for user-facing organ names: grep -rn "Consciousness\|Skepticism\|Wisdom\|Metacognition\|Principles\|Compression\|Identity\|Curiosity" static/js/*.js | grep innerHTML | grep -v "//" → count must be 0. 2) Auditor opens the Cognition surface via Ctrl+K and verifies NO organ name appears as a label. 3) Auditor verifies the human sentences are real (derived from API data, not hardcoded). 4) Auditor verifies the backend APIs are unchanged (all 8 still return 200). 5) Auditor runs the full test suite — 0 regressions.

Effort

1 day (0.5 day cognition.js rewrite + 0.5 day learn.js/today.js polish + CSS)

Build phase

Phase 1 (first — everything else builds on invisible organs)

SPEC #2 Executive Function Engine — Maestro plans, sequences, and drafts actions

V5 Principle

V5: "Maestro stops being an advisor. It becomes an operating system." Human cognition has executive function: planning, prioritization, sequencing, task inhibition, working memory, decision arbitration. Maestro currently observes and judges. V5 adds: given a judgment, Maestro produces an execution plan — schedule alignment, prepare briefing, collect evidence, recommend facilitator, draft decision memo. It changes reality, not merely understanding.

Current codebase gap

No executive function module exists. The closest is preparation.py (PreparationEngine, which prepares decisions but does not execute them). sowhat.py produces "recommended_action" as a string ("Pause non-critical work and address the incident cluster") but does not break it into steps. The gap: Maestro says WHAT to do but not HOW — no task sequence, no resource allocation, no follow-through plan.

Files to create/modify

CREATE backend/maestro_oem/executive_function.py — the ExecutiveFunctionEngine. Takes a recommendation (from sowhat.py or wisdom.py) and produces an execution plan: { steps: [{action, owner, prerequisite, estimated_time, tool}], resource_allocation: {team, budget, opportunity_cost}, follow_through: {check_in_date, success_metric, escalation_path}, drafted_artifacts: {briefing, memo, agenda} }. CREATE GET /api/oem/execute?recommendation_id=... endpoint. CREATE static/js/execute.js — a "Prepare" button on each recommendation that opens an execution plan view (not a dashboard — a calm checklist with drafted artifacts). MODIFY static/js/today.js — the "One decision" item gets a "Prepare" button that calls the executive function API.

API contract

GET /api/oem/execute?recommendation_id=rec-xxx → returns JSON: { "plan_summary": "Address the payments bottleneck by redistributing approval authority.", "steps": [{"action": "Schedule alignment meeting", "owner": "jane.d@acme.com", "prerequisite": null, "estimated_time": "30 min", "tool": "calendar"}, {"action": "Prepare briefing with evidence", "owner": "chris.t@acme.com", "prerequisite": "Schedule alignment meeting", "estimated_time": "1 hour", "tool": "docs"}], "drafted_briefing": "Team, the payments-edge bottleneck has gated 3 items for 5 days...", "follow_through": {"check_in_date": "2025-01-15", "success_metric": "Approval time < 2 days", "escalation_path": "If not resolved by Jan 15, escalate to VP Engineering"} }. Must have at least 3 steps, a drafted briefing (not empty), and a follow_through with check_in_date.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/execute?recommendation_id=. Response must have plan_summary, steps (3+), drafted_briefing (non-empty), follow_through with check_in_date. 2) Auditor verifies the steps are sequenced (prerequisites reference prior steps). 3) Auditor verifies the drafted_briefing is a real draft (not a template placeholder). 4) Auditor opens TODAY and verifies the "One decision" item has a "Prepare" button. 5) Auditor clicks "Prepare" and verifies an execution plan view appears (not a dashboard — a checklist with drafted artifacts). 6) Auditor greps executive_function.py for hardcoded steps — must find none (all derived from the recommendation + model data).

Effort

3 days (1.5 days engine + 0.5 day API + 1 day frontend execute.js + TODAY integration)

Build phase

Phase 1 (second — after organs are hidden, Maestro needs to act)

SPEC #3 — Attention Allocation — decide where attention SHOULD BE, not just where it IS

V5 Principle

V5: "Brains have limited attention. Organizations also have limited attention. Maestro should know what deserves attention, what is stealing attention, what should be ignored." Consciousness (V4) knows where attention IS. Attention Allocation (V5) decides where it SHOULD BE — and what to ignore.

Current codebase gap

consciousness.py (204 lines) tracks 7 dimensions of organizational state, including "attention" (which domains are active). But it does not allocate attention — it does not say "Engineering is over-attended; Legal is under-attended; the payments incident is stealing attention from the auth refactor." The gap: Maestro observes attention distribution but does not recommend reallocation.

Files to create/modify

CREATE backend/maestro_oem/attention.py — the AttentionEngine. Composes consciousness.py's state vector with the recommendation queue (from decision.py) and the urgency model (from sowhat.py). Produces: { current_allocation: {domain: attention_score}, recommended_allocation: {domain: attention_score}, attention_thieves: [what is stealing attention], should_ignore: [what to deprioritize], narrative: "Engineering attention is fragmented across 3 incidents. The auth refactor needs focused attention for 2 days. Consider pausing the platform-tools migration." }. CREATE GET /api/oem/attention endpoint. MODIFY static/js/today.js — replace the organizational-weather card with an attention card: "Your attention should be on X. Y is stealing attention. Consider ignoring Z."

API contract

GET /api/oem/attention → returns JSON: { "current_allocation": {"payments": 0.45, "auth": 0.20, "platform": 0.15}, "recommended_allocation": {"payments": 0.30, "auth": 0.40, "platform": 0.10}, "attention_thieves": [{"thief": "3 P1 incidents in payments", "impact": "fragmenting engineering focus", "evidence_count": 3}], "should_ignore": [{"item": "platform-tools migration", "reason": "low urgency, can wait 2 weeks", "evidence_count": 2}], "narrative": "Engineering attention is fragmented across 3 payments incidents. The auth refactor needs focused attention. Consider pausing the platform-tools migration." }. Must have at least 1 attention_thief, 1 should_ignore, and a narrative sentence.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/attention. Response must have current_allocation, recommended_allocation, attention_thieves (1+), should_ignore (1+), narrative. 2) Auditor verifies recommended_allocation differs from current_allocation (Maestro is recommending a change). 3) Auditor verifies attention_thieves reference real model data (not hardcoded). 4) Auditor opens TODAY and verifies the attention card appears (not the weather card — or the weather card is enhanced with attention allocation). 5) Auditor verifies the narrative is a sentence (not a metric dump).

Effort

2 days (1 day engine + 0.5 day API + 0.5 day frontend)

Build phase

Phase 1 (third — completes the "hide + act" foundation)

Phase 2 — Deeper Cognition (Causal, Temporal, Narrative, Forgetting, Imagination)

Phase 2 deepens the cognitive capabilities. These organs are invisible from the start (Phase 1 hid the organ names). They enhance the EXISTING organs' output without adding new visible surfaces. The user experiences deeper intelligence, not more panels.

SPEC #4 Forgetting Engine — archive zero-predictive-value events

V5 Principle

V5: "Brains forget for a reason. Compression is not forgetting. Without forgetting, memory eventually becomes noise." The Forgetting Engine identifies events that have produced zero future predictive value and archives them — removing them from the active cognitive working set while preserving them in cold storage for audit.

Current codebase gap

memory_compression.py (152 lines) compresses experience into truths/habits/mistakes. But it does not forget — all signals remain in the active evidence graph. A 2-year-old incident that has never correlated with any future outcome still occupies cognitive space. The gap: no mechanism to identify and archive zero-predictive-value events.

Files to create/modify

CREATE backend/maestro_oem/forgetting.py — the ForgettingEngine. For each learning object / law / signal in the active model, compute a "predictive_value" score based on: (1) has it been referenced in any recent recommendation? (2) has it correlated with any future outcome? (3) has it been validated/invalidated? Events with predictive_value < threshold (configurable, default 0.05) AND age > 180 days are candidates for archiving. The engine does NOT delete — it moves to a cold-storage table and removes from the active working set. CREATE GET /api/oem/forgetting endpoint (shows candidates + archived count). MODIFY backend/maestro_oem/engine.py — on ingest, check if any new signal makes an old event irrelevant (predictive_value drops) and flag it.

API contract

GET /api/oem/forgetting → returns JSON: { "archive_candidates": [{"entity_id": "...", "type": "signal", "age_days": 245, "predictive_value": 0.02, "reason": "No correlation with any future outcome in 245 days"}, ...], "archived_count": 0, "active_working_set": 1247, "threshold": 0.05, "summary": "3 events have zero predictive value and are candidates for archiving. Forgetting them will reduce cognitive noise by 0.2%." }. Must have at least 1 archive_candidate with predictive_value < 0.05 and age_days > 180 (or honestly say "no candidates yet — the organization is too young").

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/forgetting. Response must have archive_candidates array, archived_count, active_working_set, threshold, summary. 2) Auditor verifies each candidate has predictive_value < threshold and age_days > 180 (or the response honestly says "no candidates yet"). 3) Auditor verifies the engine does NOT delete (archived events are moved, not removed — check the cold-storage table exists). 4) Auditor verifies the summary is a sentence. 5) Auditor verifies no hardcoded candidates (all derived from model data).

Effort

1.5 days (1 day predictive-value scoring + 0.5 day API + cold-storage table)

Build phase

Phase 2 (build after Phase 1 — forgetting must be invisible from the start)

SPEC #5 Imagination (Counterfactual Reasoning) — "What would happen if Legal disappeared?"

V5 Principle

V5: "Organizations don't only solve today's problems. They imagine futures." The Imagination organ performs counterfactual reasoning: given a hypothetical change (Legal disappeared, Engineering doubled, Revenue halved, Acquisition occurred), predict the organizational impact using causal models (Spec #6) and historical data.

Current codebase gap

digital_twin.py (743 lines) runs scenarios (person leaves, team doubles, meetings cut, hires added) via the DigitalTwinSimulator. But these are PARAMETRIC scenarios — they adjust inputs and compute outputs. They are not COUNTERFACTUAL — they do not reason about causality ("Legal disappeared BECAUSE of the reorg, and the reorg also affected Engineering, so the impact is..."). The gap: no causal counterfactual reasoning.

Files to create/modify

CREATE backend/maestro_oem/imagination.py — the ImaginationEngine. Takes a counterfactual ("What would happen if Legal disappeared?") and: (1) identifies the causal dependencies of Legal (from evidence_graph.py + coordination.py), (2) retrieves historical analogues (has any team ever disappeared or been restructured?), (3) simulates the impact using digital_twin.py, (4) produces a causal narrative: "If Legal disappeared, 3 consequences would follow: (a) OAuth reviews would stall (Legal is the sole reviewer), (b) Contract turnaround would drop to zero (no legal capacity), (c) Engineering velocity would increase 15% (fewer review gates). Historical analogue: the Q2 legal-team reorg produced similar dynamics." CREATE GET /api/oem/imagine?scenario=... endpoint. MODIFY static/js/ask_v2.js — when the user asks a "what if" question, route to the Imagination engine.

API contract

GET /api/oem/imagine?scenario=What+would+happen+if+Legal+disappeared → returns JSON: { "scenario": "Legal disappeared", "consequences": [{"effect": "OAuth reviews would stall", "cause": "Legal is the sole reviewer for auth-service PRs", "evidence_count": 5, "confidence": 0.82}, {"effect": "Contract turnaround would drop to zero", "cause": "No legal capacity exists outside the team", "evidence_count": 3, "confidence": 0.91}, {"effect": "Engineering velocity would increase ~15%", "cause": "Fewer review gates", "evidence_count": 2, "confidence": 0.58}], "historical_analogue": "The Q2 legal-team reorg produced similar dynamics: review time dropped 40%, then incidents increased 20% within 3 weeks.", "narrative": "If Legal disappeared, the immediate effect would be faster engineering (fewer gates), but the secondary effect would be more incidents (unreviewed changes). The historical analogue suggests the gains would reverse within 3 weeks." }. Must have at least 2 consequences with cause + evidence_count + confidence, and 1 historical analogue.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/imagine?scenario=What+would+happen+if+Legal+disappeared. Response must have consequences (2+), historical_analogue (non-empty), narrative (non-empty). 2) Auditor verifies each consequence has cause + evidence_count + confidence. 3) Auditor verifies the historical_analogue references real model data. 4) Auditor opens ASK v2 and asks "What would happen if Engineering doubled?" — verifies the response includes counterfactual reasoning (not just keyword search). 5) Auditor verifies the narrative is a causal story (not a metric dump).

Effort

2 days (1.5 days counterfactual engine + 0.5 day API + ASK v2 integration)

Build phase

Phase 2 (build after Causal Cognition #6 — imagination requires causal models)

SPEC #6 — Causal Cognition — "A caused B because 5 interventions produced the same sequence"

V5 Principle

V5: "Today: A correlates with B. Future: We believe A caused B because five similar interventions produced the same sequence." Causal cognition moves Maestro from correlation to causation. The evidence graph tracks co-occurrence; causal cognition tracks intervention-outcome sequences.

Current codebase gap

evidence_graph.py tracks evidence nodes and edges (co-occurrence). pattern.py detects patterns (regularities). But neither tracks CAUSALITY — "X intervention produced Y outcome, 5 times, in the same sequence." The gap: Maestro can say "bottlenecks correlate with velocity drops" but not "removing the bottleneck CAUSED the velocity to recover, because 3 interventions produced the same recovery sequence."

Files to create/modify

CREATE backend/maestro_oem/causal.py — the CausalEngine. Scans the prediction_lifecycle.py database for resolved predictions where an intervention was recommended and an outcome was observed. Builds a causal chain: {intervention → outcome, confidence, sequence_count, failed_count}. A causal claim requires: (1) the intervention preceded the outcome, (2) the same intervention-outcome sequence occurred >= 3 times, (3) the outcome did not occur without the intervention (control). CREATE GET /api/oem/causal?intervention=... endpoint. MODIFY backend/maestro_oem/wisdom.py — when synthesizing judgment, cite causal chains (not just correlations) where available. MODIFY static/js/cognition.js — the wisdom section now shows "Why we believe this" with causal evidence (not just correlation).

API contract

GET /api/oem/causal?intervention=redistribute+approval+authority → returns JSON: { "intervention": "Redistribute approval authority", "causes": [{"effect": "Approval time drops below 2 days", "sequence_count": 3, "failed_count": 0, "confidence": 0.91, "first_observed": "2024-03-15...", "evidence": ["rec-xxx where bottleneck was addressed → velocity recovered within 5 days", "rec-yyy where gate was redistributed → approval time dropped 60%"]}], "narrative": "Redistributing approval authority causes approval time to drop below 2 days. This has been observed 3 times with 0 failures. The causal chain is strong." }. Must have at least 1 cause with sequence_count >= 3 (or honestly say "insufficient intervention data — the pilot has not produced enough resolved predictions for causal inference").

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/causal?intervention=redistribute+approval+authority. Response must have causes array (or honest "insufficient data" message). 2) If causes exist, each must have sequence_count >= 3, failed_count, confidence. 3) Auditor verifies the narrative is a causal claim (not a correlation). 4) Auditor opens the Cognition surface and verifies the Wisdom section cites causal evidence where available. 5) Auditor verifies no hardcoded causal claims (all derived from prediction_lifecycle data).

Effort

2 days (1.5 days causal-inference engine + 0.5 day API + wisdom.py integration)

Build phase

Phase 2 (build before Imagination #5 — counterfactuals require causal models)

SPEC #7 — Temporal Trajectories (Org-wide) — "Trust has fallen slowly for 8 weeks"

V5 Principle

V5: "Don't remember snapshots. Remember trajectories." Today, time_axis.py works for one domain. V5 extends temporal trajectory memory to ALL organizational dimensions (trust, attention, energy, conflict, knowledge mobility, etc.). Every metric becomes a trajectory, not a point.

Current codebase gap

time_axis.py (167 lines) returns past/present/future for a single domain (e.g., "payments"). consciousness.py (204 lines) returns a point-in-time state vector (attention, knowledge, trust, etc. at this moment). The gap: no org-wide trajectory memory. Maestro cannot say "trust between Engineering and Legal has fallen slowly for 8 weeks" — it can only say "trust is currently 0.42." The trajectory — the trend over time — is not tracked for organizational dimensions.

Files to create/modify

CREATE backend/maestro_oem/temporal_trajectories.py — the TemporalTrajectoryEngine. Maintains a rolling 90-day trajectory for each consciousness dimension (attention, knowledge, trust, conflict, energy, uncertainty, learning). On each signal ingest, updates the trajectory. Produces: { dimension: {current, trend, slope, duration, narrative} }. CREATE GET /api/oem/trajectories endpoint. MODIFY backend/maestro_oem/consciousness.py — enrich each state dimension with its trajectory (not just current value). MODIFY static/js/today.js — the attention card (Spec #3) now shows trajectories: "Trust between Engineering and Legal has fallen for 8 weeks."

API contract

GET /api/oem/trajectories → returns JSON: { "trust": {"current": 0.42, "trend": "declining", "slope": -0.03, "duration_weeks": 8, "narrative": "Trust between Engineering and Legal has fallen slowly for 8 weeks. The decline started after the OAuth consolidation disagreement."}, "attention": {"current": 0.65, "trend": "stable", "slope": 0.01, "duration_weeks": 12, "narrative": "Attention distribution has been stable for 12 weeks."}, ... 7 dimensions ... }. Must have at least 3 dimensions with trend != "stable" (or honestly say "insufficient history for trend analysis — need 4+ weeks of data").

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/trajectories. Response must have at least 3 dimensions with current, trend, slope, duration_weeks, narrative. 2) Auditor verifies each narrative is a trajectory sentence (not a point-in-time metric). 3) Auditor verifies the trajectory is computed from real signal history (not hardcoded). 4) Auditor opens TODAY and verifies the attention card references trajectories (not just current state). 5) Auditor verifies dimensions with < 4 weeks of data return "insufficient history" honestly.

Effort

1.5 days (1 day trajectory-tracking engine + 0.5 day API + consciousness.py + today.js integration)

Build phase

Phase 2 (build after Attention Allocation #3 — trajectories enrich attention decisions)

Phase 3 — Ambient Integration + Institutional Memory

Phase 3 is the V5 end-state. Maestro stops being a destination. It appears in the user's existing tools — GitHub, Jira, Slack, Zoom, Outlook. The cognitive organs are invisible. The user experiences Maestro as a calm companion that arrives with the right context, prepares work before it's requested, and helps the organization make better decisions — without requiring anyone to learn the architecture behind it.

SPEC #8 Institutional Memory Recall — "When have we been here before?"

V5 Principle

V5: "Eventually Maestro owns the organization's autobiography. Not CRM. Not Slack history. Not documents. Its autobiography." Institutional Memory Recall answers "When have we been here before?" with 3 moments + lessons — not documents, not search results, but organizational memories.

Current codebase gap

The closest existing capability is the autocomplete engine (semantic-ish retrieval from OEM state). But autocomplete returns suggestions, not memories. learning.py tracks calibration but not narrative recall. memory_compression.py compresses but does not retrieve by situational similarity. The gap: no mechanism to take a current situation ("we're about to delay a launch for Legal review") and retrieve the 3 most similar past moments with their outcomes and lessons.

Files to create/modify

CREATE backend/maestro_oem/institutional_recall.py — the InstitutionalRecallEngine. Takes a situation description (from the current recommendation or the user's ASK query) and retrieves the top 3 most similar past moments from the learning database + signal history + resolved predictions. For each moment: {when, what_happened, what_we_did, what_we_learned, outcome}. Uses embedding similarity (or keyword overlap if no embedding model) against the historical signal + learning object + prediction database. CREATE GET /api/oem/recall?situation=... endpoint. MODIFY static/js/ask_v2.js — when the user asks a question, append a "When you've been here before" section with up to 3 recalled moments.

API contract

GET /api/oem/recall?situation=delaying+launch+for+Legal+review → returns JSON: { "moments": [{"when": "2024-08-15", "situation": "Q3 auth launch delayed 3 days for Legal review", "what_we_did": "Proceeded with Legal review. Launch succeeded. Post-launch bugs were 27% lower than average.", "what_we_learned": "Legal review before launch reduces post-launch incidents.", "outcome": "succeeded", "evidence_count": 5}], "summary": "You've been in a similar situation 3 times. In 2 of 3, proceeding with the Legal review led to better outcomes. In 1, the delay caused a missed deadline." }. Must have at least 1 moment with when, situation, what_we_did, what_we_learned, outcome (or honestly say "this situation is novel — no similar past moments found").

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/recall?situation=delaying+launch+for+Legal+review. Response must have moments array (1+) or honest "novel situation" message. 2) If moments exist, each must have when, situation, what_we_did, what_we_learned, outcome. 3) Auditor verifies the moments reference real historical data (not hardcoded). 4) Auditor opens ASK v2 and asks a question — verifies a "When you've been here before" section appears (if similar moments exist). 5) Auditor verifies the summary is a narrative (not a metric dump).

Effort

2 days (1.5 days similarity-retrieval engine + 0.5 day API + ASK v2 integration)

Build phase

Phase 3 (build after Phase 2 — recall requires temporal + causal + narrative cognition)

SPEC #9 — Ambient Integration (Chrome Extension) — Maestro appears in GitHub/Jira/Slack/Zoom/Outlook

V5 Principle

V5: "The user should almost never visit Maestro. Maestro visits the user." The Chrome extension is the V5 end-state. It detects which tool the user is in (GitHub PR page, Jira issue, Slack channel, Zoom meeting, Outlook email) and injects a calm Maestro contextual card with the right intelligence for that context. No dashboard. No app to open. The cognition appears where the work happens.

Current codebase gap

The maestro-ambient-extension/ directory exists in the repository (from V2) but contains only a stub. The WORK surface (work.js) displays AMBIENT CARDS about what Maestro sees in each tool — but the user must open Maestro to see them. The gap: no actual browser extension that injects Maestro into GitHub/Jira/Slack/Zoom/Outlook. The cognition is a destination, not a companion.

Files to create/modify

CREATE maestro-ambient-extension/manifest.json — Chrome extension manifest (Manifest V3). CREATE maestro-ambient-extension/content.js — content script that detects the current page (GitHub PR, Jira issue, Slack channel, Zoom meeting, Outlook email) and injects a Maestro card. CREATE maestro-ambient-extension/background.js — service worker that calls the Maestro API (/api/oem/sowhat, /api/oem/recall, /api/oem/attention) with the page context. CREATE maestro-ambient-extension/styles.css — calm, unobtrusive card styling (max 2 sentences, dismissible). The extension should: (a) on GitHub PR pages, show "You've solved this before. Review?" (from institutional recall), (b) on Jira issues, show "Platform already attempted this. Read before starting?" (from pattern detection), (c) on Zoom meetings, show "Procurement joins in 2 minutes. Lead with ROI." (from consciousness state), (d) on Outlook emails, show "Legal champion changed yesterday. Proposal should adapt." (from identity drift).

API contract

No new Maestro API. The extension calls existing endpoints. The acceptance test is: install the extension in Chrome, navigate to a GitHub PR page, verify the Maestro card appears with real data (not a placeholder). The card must be dismissible. The card must not appear on non-supported pages. The extension manifest must request only the minimum permissions (activeTab, storage, host permissions for the Maestro server).

Acceptance test (auditor will run)

1) Auditor installs the extension in Chrome (load unpacked from maestro-ambient-extension/). 2) Auditor navigates to a GitHub PR page and verifies a Maestro card appears with real data (not a placeholder). 3) Auditor verifies the card is dismissible (clicking X removes it). 4) Auditor verifies the card does NOT appear on non-supported pages (e.g., google.com). 5) Auditor checks the manifest — must request only activeTab, storage, and host permissions (no broad permissions). 6) Auditor navigates to a Jira issue and verifies a different card appears (context-aware). 7) Auditor verifies the card calls the Maestro API (network tab shows /api/oem/ requests).

Effort

5 days (2 days Chrome extension scaffolding + 2 days context detection + API integration for 3 tools + 1 day Jira/Slack/Zoom/Outlook pattern matching)

Build phase

Phase 3 (capstone — requires all Phase 1 + 2 capabilities to be working)

Build Order and Dependencies

Phase	Specs	Duration	Why This Order	Unlocks
1	#1 Hide Organs + #2 Executive Function + #3 Attention	~6 days	Foundation. Hide what exists (V4 organs visible). Add the ability to act (execution). Decide	Invisible organs + action capability + attention. Decide
2	#4 Forgetting + #5 Imagination + #6 Causal + #7 Temporal	~7 days	Deeper cognition. Forgetting reduces noise. Imagination enables counterfactuals. Causal + temporal	Clean memory + counterfactuals + causal + temporal
3	#8 Institutional Recall + #9 Ambient Extension	~7 days	End-state. Institutional recall answers "when have we done this before?" The Chrome extension	Organizational memory + ambient companion
TOTAL	**9 specs**	**~20.5 days**	**V5: from cognitive architecture to invisible cognitive layer**	**The invisible layer**

Phase 1 is the V5 foundation. Before adding ANY new cognitive capability, the existing organs must be hidden. Spec #1 (hide organ names) is 1 day and must be done first. Every subsequent spec must be invisible from the start — no new visible panels, no new organ names in the UI. The V5 constitutional law: "Every release must make Maestro feel simpler."

The V5 Litmus Test

EVERY RELEASE MUST MAKE MAESTRO FEEL SIMPLER

The V5 constitutional law: "Every release must make Maestro feel simpler, even if it becomes dramatically more intelligent internally." This is the inverse of every previous constitution. V2/V3/V4 added visible capability. V5 removes visible complexity. The internal intelligence doubles; the visible UI shrinks.

The V5 acceptance test for every spec: Does this spec REDUCE the visible UI? If yes, it is V5. If it adds a new panel, a new page, a new organ name, a new metric — it is NOT V5, regardless of how intelligent the backend is. The 9 specs above all pass this test:

• Spec #1 (Hide Organ Names): REMOVES 8 organ-name labels from the UI. Net UI reduction.

• Spec #2 (Executive Function): adds a "Prepare" button (1 element) but replaces the need to manually plan (removes cognitive load). Net simplification.

• Spec #3 (Attention): REPLACES the weather card with an attention card (1-for-1 swap, higher value). Net neutral UI, higher intelligence.

• Spec #4 (Forgetting): zero UI change (backend-only archival). Net UI unchanged, intelligence ↑.

• Spec #5 (Imagination): enhances ASK v2 (existing surface). No new panel. Net UI unchanged, intelligence ↑.

• Spec #6 (Causal): enhances Wisdom (existing organ output). No new panel. Net UI unchanged, intelligence ↑.

• Spec #7 (Temporal): enhances consciousness (existing organ output). No new panel. Net UI unchanged, intelligence ↑.

• Spec #8 (Institutional Recall): enhances ASK v2 (existing surface). No new panel. Net UI unchanged, intelligence ↑.

• Spec #9 (Ambient Extension): REMOVES the need to open Maestro. The UI moves OUT of Maestro and INTO the user's tools. Net UI reduction in Maestro itself.

If any spec fails this test, redesign it. The V5 bar is not "does it work?" but "does it make Maestro feel simpler?" That is a different standard. Build to it.

The V4 → V5 Transition

Before building V5 specs, the coder must close the V4 quality issues from round 16. These are small (~4 hours) and must be done first — V5 specs build on V4 organs, so the V4 organs must be quality-defect-free.

V4 Gap	Fix	Effort
Curiosity: nested quotes from embedding rec titles as beliefs	Use actual assumptions from assumption.py, not rec titles. Generate domain-level questions.	2 hours

Skepticism: same nested-quote issue	Same fix as Curiosity — use actual assumptions.	1 hour
Time-axis hardcoded to "engineering" (404s with demo seed)	Derive domain from data or use "payments"/"auth" which work.	30 min
Field-name mismatches (category/evidence_count vs type/evidence)	Add aliases or update spec to match implementation.	30 min
TOTAL V4 cleanup before V5		**~4 hours**

Sequence: V4 cleanup (~4 hours) → V5 Phase 1 (~6 days) → V5 Phase 2 (~7 days) → V5 Phase 3 (~7 days). Total: ~20.5 days + 4 hours. When complete, Maestro is The Invisible Layer — a cognitive system that doubles internal intelligence while shrinking the visible UI, until the user almost never visits Maestro. Maestro visits them.

The Recurring Pattern — Final Warning for V5

THE PATTERN WAS BROKEN IN ROUND 16. DO NOT LET V5 REINTRODUCE IT.

Round 16 was the first time in 16 rounds that the "built but not applied" pattern did NOT recur. All 8 V4 organs were built AND wired AND user-visible. The 5-point checklist was followed. The CI pipeline caught test failures. The acceptance tests checked application. The pattern broke.

V5 reintroduces a new risk: the "added visibility" pattern. V5 specs must REDUCE the visible UI. But the natural tendency when building a new capability is to add a panel for it. Spec #2 (Executive Function) wants a "Prepare" button — that is 1 element added, but it must replace manual planning (cognitive load removed). Spec #5 (Imagination) wants to enhance ASK v2 — it must NOT add a new "Imagination" panel. Spec #8 (Institutional Recall) wants to enhance ASK v2 — it must NOT add a new "Recall" surface. Every V5 spec must be checked against the litmus test: "Does this make Maestro feel simpler?"

The V5 5-point checklist (updated from V3/V4):

1. Ran the FULL acceptance test (API + frontend)?

2. Checked APPLICATION (frontend calls the API), not EXISTENCE?

3. Ran the FULL test suite (not a subset)?

4. Verified with a LIVE API call?

5. Checked the user-facing UI — and verified it is SIMPLER, not more complex?

Point 5 is new for V5. The check is not just "does the UI work?" but "is the UI simpler than before?" If a V5 spec adds visible complexity, it fails the litmus test — even if the backend is brilliant. Redesign it to be invisible. The V5 bar is UI reduction, not capability addition.

The CI pipeline will catch test failures. The acceptance tests will catch built-but-not-applied. The litmus test will catch added-visibility. Do not let V5 reintroduce complexity. The Invisible Layer means invisible.

Final Charge to the Coder

V4 proved that an organization can have cognitive organs. V5 makes those organs disappear. The user should almost never visit Maestro. Maestro visits the user — in Outlook, in Zoom, in GitHub, in Jira, in Slack. The cognitive organs become internal. The UI becomes calm. The product stops being a destination and becomes a companion.

The V5 litmus test for every commit: "Does this make Maestro feel simpler, even if it becomes more intelligent internally?" If yes, ship it. If it adds visible complexity, reject it. V5 is the first constitution whose primary metric is UI reduction.

Build order: V4 cleanup → Phase 1 (#1-#3) → Phase 2 (#4-#7) → Phase 3 (#8-#9). ~20.5 days. When complete, Maestro is The Invisible Layer — a cognitive system that doubles internal intelligence while shrinking the visible UI, until the user almost never opens Maestro. The organs are internal. The actions are real (executive function). The attention is allocated. The memory forgets. The imagination counterfactuals. The causality is real. The trajectories are temporal. The recall is institutional. And the Chrome extension makes it all appear where the work happens.

The end state is not an enterprise platform. It is not a dashboard. It is not a copilot. It is the organization's autobiography — accumulated judgment, impossible to recreate, more valuable than its CRM. It answers the question no existing software answers: "Given everything this organization has lived through, how should it think now?" And it answers it not in a dashboard, but in a calm whisper that appears in the tool where the user is already working. That is The Invisible Layer. Build it.